

KN-LIO: Geometric Kinematics and Neural Field Coupled LiDAR-Inertial Odometry

Anonymous Authors

Abstract—Recent advancements in LiDAR-Inertial Odometry (LIO) have enhanced various robotic-related applications. However, traditional LIO systems tend to focus more on localization rather than mapping, with maps consisting mostly of sparse geometric elements, which is not ideal for downstream tasks. Recent emerging neural field technology has great potential in dense mapping, but pure LiDAR mapping is difficult to work on high-dynamic vehicles. To mitigate this challenge, we present a new solution that tightly couples geometric kinematics with neural fields to enhance simultaneous state estimation and dense mapping capabilities. We propose both semi-coupled and tightly coupled Kinematic-Neural LIO (KN-LIO) systems that leverage online SDF decoding and iterated error-state Kalman filtering to fuse laser and inertial data. Our KN-LIO minimizes information loss and improves accuracy in state estimation, while also accommodating asynchronous multi-LiDAR inputs. Evaluations on diverse high-dynamic datasets demonstrate that our KN-LIO achieves performance on par with or superior to existing state-of-the-art solutions in pose estimation and offers improved dense mapping accuracy over traditional pure LiDAR methods. The relevant code and datasets will be made available to the robotics community upon publication.

Index Terms—LiDAR-Inertial Odometry, Dense Mapping, Neural Field, SLAM

I. INTRODUCTION

ROBOTICS is gradually becoming integrated into daily human life. Their interaction with the environment and humans relies on a comprehensive perception of both intro-state and extro-state, which is facilitated by SLAM (Simultaneous Localization and Mapping) technologies. Recent rapid advancements in such techniques based on the fusion of LiDAR (Light Detection and Ranging) and IMU (Inertial Measurement Unit), namely the LiDAR-Inertial Odometry (LIO), have significantly propelled the implementation of related applications.

Early LIO systems focused on a loosely coupled integration of these two information sources [1]. While this way is straightforward, it does not fully leverage the complementary information between them. Recent research on LIO systems reveals that a tightly coupled fusion is the key to maximizing the complementary characteristics of the two types of sensors [2]–[6]. On one hand, real-time estimation of IMU biases enhances the accuracy of state prediction and helps eliminate motion distortion in point clouds. On the other hand, the registration of the undistorted point clouds with the environmental map can effectively suppress system state drift, enabling the system to work stably through the precise collaboration of LiDAR and IMU. However, existing LIO solutions tend to prioritize localization over mapping. In other words, their map representations often consist of geometric elements that facilitate

pose estimation, such as points, lines, surfels, or voxels. Yet, such geometric elements are not what downstream tasks like navigation, augmented reality, and virtual reality aspire to. In practice, such applications prefer a dense three-dimensional scene representation rather than sparse geometric elements.

Traditional dense reconstruction techniques can be categorized into offline reconstruction based on full measurements and online incremental reconstruction. The former includes methods such as Poisson reconstruction [7], [8], which aims to fit surface equations, and graph-cut-based surface reconstruction [9], which focuses on identifying segmentation surfaces. The latter first emerged in RGBD-SLAM [10], where it partitions the scene into regular grids and establishes a distance field from spatial points to their nearest surfaces, employing the marching cube algorithm for dense scene reconstruction [11]. In addition to the aforementioned traditional methods, dense reconstruction techniques for point clouds have recently achieved promising results driven by advancements in machine learning, especially with the rise of neural radiance fields (NeRF) [12]. In offline reconstruction, some approaches represent the scene as a compact octree and learn features attached to the vertices of spatial grids to fit dense neural fields [13], [14]. For online reconstruction, methods based on octree grid representation [15] and utilizing sparse neural point representation [16], both learn SDF (Signed Distance Function) decoders from scene features, track LiDAR poses, and jointly learn map neural features on the fly. Compared to traditional methods, the online solutions based on neural field representations not only enjoy the advantage of constructing continuous fields at arbitrary resolutions but also more naturally leverage data-driven approaches to fully integrate both introspective and extrospective perception information, thereby producing more accurate scene maps.

However, despite the commendable performance of such pure LiDAR-focused neural dense reconstruction techniques in relatively stable near-2D autonomous driving scenarios, their robustness and precision in more challenging cases, such as handheld devices, backpacks, or fast 3D drones, remain unsatisfactory. A straightforward idea is to enhance online reconstruction under such complex motion conditions by integrating the instantaneous dynamic perception capability of IMUs. However, extensive research on traditional LIO has shown that loosely coupled approaches do not significantly improve system performance. While the tight coupling of IMU kinematics with LiDAR point cloud geometric features has been extensively studied, how to couple geometric kinematics with neural fields remains an unsolved and challenging problem. The primary reasons lie in two aspects. On one

hand, neural fields are encoded by high-dimensional neural features, while IMU-based geometric kinematics evolves on a composite manifold; on the other hand, IMU-based geometric kinematics have explicit mathematical expressions, whereas neural fields are implicitly represented within decoders and neural features.

To tackle these challenges, we represent the scene map as a neural point cloud and obtain the distance to the nearest surface through online learning of an SDF decoder and neural features, while fusing laser and inertial observations based on an error-state Kalman filter. To effectively estimate the system state, we propose both semi-coupled and tightly coupled LiDAR-Inertial Odometry systems that integrate geometric Kinematics with Neural fields, Semi-KN-LIO and KN-LIO for short, respectively. In the semi-coupled approach, we first predict the system state (position, velocity, orientation, IMU biases) using high-frequency IMU readings, then undistort the input point cloud and register it to the built map based on the predicted state, and finally update the system error state using the registered pose as a composite measurement. This semi-coupled method allows for a relative decoupling of geometric kinematics and neural fields while utilizing the neural field observations to correct the system's error state. In the tightly coupled one, we further integrate the error state updates with the point cloud registration process, obtaining the Kalman gain by formulating the Kalman measurement model from residuals of the incoming point cloud to the built neural field map in each iteration to update the system's error state. Although this tightly coupled method is more complex, it reduces information transfer loss and effectively enhances the accuracy of state estimation.

In addition, we explore two critical issues that impact the effectiveness of incremental neural dense mapping. The first issue pertains to the integration of multi-LiDAR data to enhance map completeness. As is widely recognized, the vertical resolution of rotating mechanical LiDAR is considerably lower than its horizontal resolution, while solid-state LiDAR is limited to providing point cloud observations within a forward-facing field of view. Consequently, the fusion of non-synchronous or even heterogeneous multi-LiDAR point clouds to construct a more comprehensive map represents a pressing challenge at the current stage. In this work, benefiting from the fact that point cloud registration based on neural point representation does not require establishing point-to-point (or feature-to-feature) correspondences, we extend the system to accommodate non-synchronous multi-LiDAR inputs. By leveraging sensor extrinsic parameters and the timestamps associated with each observation, we transform non-synchronous multi-LiDAR point clouds (whether homogeneous or heterogeneous) into a unified fused point cloud. Additionally, we utilize online IMU propagation to remove motion distortion from the point cloud, thereby enabling our Semi-KN-LIO and KN-LIO to seamlessly handle both single-LiDAR and multi-LiDAR data without distinction. This approach effectively exploits the complementary advantages of multiple viewpoints, facilitating the construction of more complete maps and enhancing the accuracy of state estimation.

The second issue is the development of a reasonable and

efficient nearest-neighbor query method. In neural point-based map optimization, it is necessary to frequently query neighboring neural points to construct SDF losses for iteratively optimizing the associated features. Thus, an efficient query method is crucial for the learning efficiency of the system. Traditionally, this is achieved by constructing a global hash table for the whole map neural points. However, this approach requires frequent hash mapping calculations and handling of potential hash conflicts during localization and mapping. In this paper, we propose an alternative to avoid these operations. Specifically, as the carrier moves, we dynamically maintain a local 3D index table based on the local map range, where each grid stores the index of the neural point located within it. This way, during each query, the index of a neural point can be directly obtained from its coordinates, significantly speeding up the query process and eliminating the need for hash checks.

We evaluated our systems on a large number of datasets gathered from high-dynamic platforms, including handheld devices, mobile robotics, and drones. The results demonstrate that, in terms of pose estimation, KN-LIO achieves performance comparable to existing state-of-the-art solutions. In terms of dense mapping, KN-LIO yields more accurate results than pure LiDAR mapping solutions.

To summarize, our contributions are fourfold:

- To our knowledge, we are the first to study the coupling of geometric kinematics with neural fields for achieving high-precision LiDAR-Inertial odometry and incremental dense mapping. Our KN-LIO achieves pose tracking performance comparable to state-of-the-art LIO methods while producing superior results in dense mapping.
- We also propose a semi-coupled approach to fuse the geometric motion information provided by IMU data with the registration information between point clouds and neural fields. Compared to pure LiDAR-based methods, this approach enhances the robustness and accuracy of the system.
- Our system supports non-synchronous and heterogeneous multi-LiDAR inputs, effectively fusing complementary point cloud information from diverse perspectives to enhance map completeness and pose estimation accuracy.
- We propose a more direct nearest neighbor point query method for neural point maps, eliminating the need for hash mapping and hash collision checking, thereby improving mapping query efficiency.

II. RELATED WORK

A. LiDAR-Inertial Odometry

In recent years, the effective fusion of LiDAR and IMU has significantly improved the performance of LiDAR-Inertial Odometry/SLAM systems for localization and mapping. Existing approaches can be divided into feature-based ones and direct ones depending on whether geometric features are extracted. Feature-based approaches often rely on LOAM [17] features. Initially, these methods drew inspiration from valuable experiences in visual-inertial SLAM. For example, based on VINS-Mono [18], LIO-Mapping [19] utilizes graph optimization to construct geometric constraints using the line

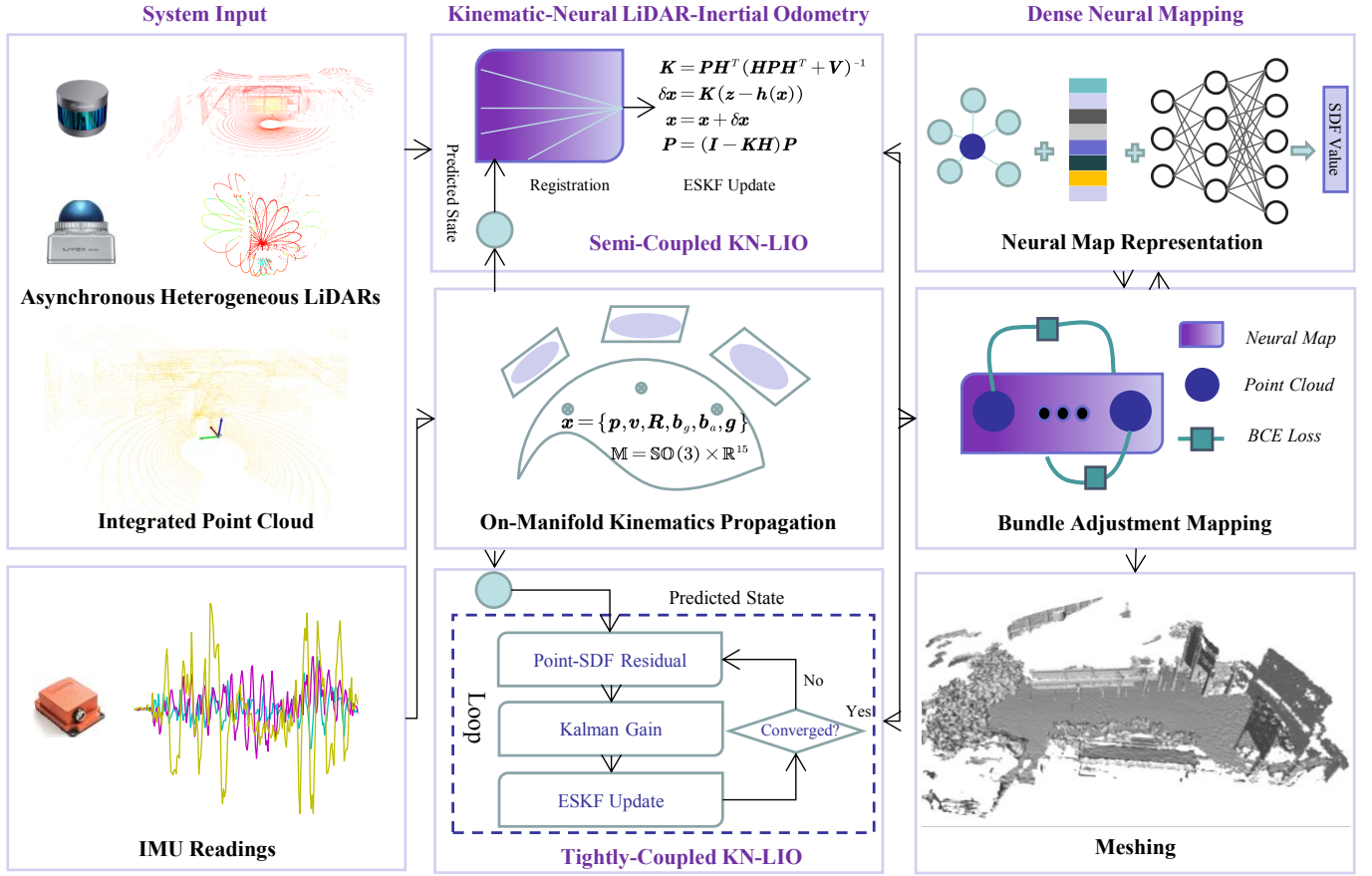


Fig. 1: Overview of Kinematic-Neural LiDAR-Inertial Odometry. Left: The high-frequency IMU readings are used for recursive system state propagation on the manifold. Asynchronous heterogeneous point clouds are first integrated based on timestamps and extrinsics, and then de-skewed according to the recursive state. Middle: In a semi-coupled mode, the de-skewed point cloud is registered with a neural field by minimizing point-SDF residuals, and the error-state Kalman filter is updated with the registration result. In a tightly coupled mode, the system calculates the point-SDF residuals and updates the error-state Kalman filter based on them, iterating until convergence. Right: The system represents the map as a set of neural points, where the feature of a given spatial point is obtained by weighted distance to its neighboring neural points, and its corresponding SDF value is decoded from its features through a simple MLP. Under the tracked state, the system performs bundle optimization on the local neural map together with the current point cloud. Finally, the dense mesh of the scene is reconstructed through SDF queries and marching cubes.

and plane features defined by LOAM [17], making it the first tightly coupled LiDAR-Inertial odometry solution. Subsequently, Qin *et al.* proposed LINS [2], which achieves effectively tightly coupling of feature-based ICP registration [20] and Kalman filter [21]. Shan *et al.* introduced LIO-SAM [3], which abandons inter-frame registration, aligns point clouds to local maps using LeGO-LOAM [22] features, and jointly registers pose and IMU pre-integration based on smooth and mapping techniques [23], [24] to optimize system state. The reasonable utilization of line/plane features and effective estimation of IMU biases enabled LIO-SAM to achieve impressive results at the time, but its design requires high performance from the IMU and performs moderately in complex and unstructured scenarios. Additionally, CLINS [25] fuses LiDAR-Inertial data in a continuous-time framework, transforming the state estimation problem into a joint optimization problem of continuous-time trajectories and observed features, effectively fusing information from multiple frames but with higher computational complexity.

In the domain of direct methods, early works were based on probabilistic grid maps, loosely integrating LiDAR and IMU information [1]. Building upon this representation, Wang *et*

al. drew inspiration from LIO-SAM and proposed the tightly coupled direct approach D-LIOM [4]. Furthermore, to combine laser-visual-inertial information, Wang *et al.* introduced Ct-LVI [26] within a continuous-time SLAM framework, constructing a tightly coupled LiDAR-Inertial subsystem Ct-LIO through direct registration. In a similar way, Chen *et al.* [5] improved upon the direct LiDAR odometry system (DLO) by tightly coupling state estimation based on GICP in a coarse-to-fine manner, achieving satisfactory performance in various complex scenarios. Building on their earlier work Fast-LIO [27], Xu *et al.* no longer extract point cloud line features, but directly construct point-to-plane constraints by fitting surfaces to neighboring points queried in an ikdtree, proposing Fast-LIO2 [6], which has achieved state-of-the-art performance in terms of localization accuracy and robustness.

B. Dense Mapping

Traditional dense mapping methods can be classified into Poisson-based, tetrahedron-based, voxel-based, and facet-based. Poisson-based reconstruction [7], [8] represents the scene surface as an indicator function and fits the target object/scene surface gradients (in the normal vectors of the

point cloud) through optimization techniques to reconstruct the surface. The surface is then extracted from this function using techniques similar to the Marching Cube algorithm [11]. This implicit representation gives the reconstructed scene continuous, compact, and excellent characteristics. Tetrahedron-based methods [9] first decompose the point cloud into tetrahedra, then perform binary classification on these decomposed tetrahedra, and finally extract the surface from their labels. Voxel-based methods [10] first appeared in online reconstruction from RGB-D data, aiming to accelerate the mapping efficiency using GPU-accelerated regularized spatial division. This technology estimates the truncation distance from the observation point cloud to the nearest surface of the divided grid points and extracts the surface from the grid using Marching Cube [11]. Although GPU acceleration can achieve real-time performance in indoor scenes, challenges in storage space and long-term mapping consistency still exist when effectively extending it to large outdoor scenes [28]–[31]. Additionally, some methods represent the scene as a series of facets/points and reconstruct the surface through sampling and point rendering techniques [32], [33]. This dispersed representation approach is more flexible than representing the entire function or regular grid, but reconstructing high-precision surfaces requires more elements, inevitably increasing computational and storage burdens. Recently, Lin et al. [34] proposed an efficient facet-based mesh reconstruction algorithm, achieving real-time online mesh reconstruction.

The significant advancements in machine learning techniques have also promoted learning-based surface reconstruction, especially with the emergence of neural radiance fields in recent years. To achieve novel view synthesis, Mildenhall *et al.* [12] represent the scene as an implicit function with position and orientation as input parameters, estimating the view color through ray tracing. Similarly, the depth of spatial points can also be obtained from the opacity in the scene function through ray tracing. It can be said that this implicit representation of the scene aligns well with Poisson reconstruction. Using such representation, a series of works have made active attempts at online SLAM based on NeRF [35], [36], but the low rendering efficiency has always been a bottleneck. To accelerate the rendering procedures, some methods divide space into grids and characterize the scene with features attached to grid vertices, decoding target values with an MLP [13], [14], [37]. There are also approaches that represent the scene as neural point clouds [38]–[42]. These grid and point representations were initially focused on object-level or room-level reconstruction. In order to extend them to outdoor scenes, many works have made active explorations, such as structuring sparse grids based on octrees [15] or organizing neural points based on hash grids [39]. It is evident that these approaches cleverly draw on the development path of traditional geometric methods while effectively empowered by the advantages of data-driven learning.

III. METHOD

A. Overview

The system workflow is illustrated in Fig. 1. We represent the scene as a set of learnable neural points, where the SDF

value at each spatial position is encoded by the weighted distance from its surrounding neural points' features and estimated by a simple MLP decoder. For the asynchronous heterogeneous LiDAR input, we first generate an integrated point cloud based on spatiotemporal parameters. For the high-frequency IMU data, we recursively estimate the nominal state and error state, as well as the corresponding covariance, using an error-state Kalman filter (ESKF), and remove point cloud distortions based on the propagated state. Subsequently, within the semi-coupled KN-LIO, we register the de-skewed point cloud to the current neural map by minimizing the SDF residuals of sampled points to obtain the registration pose, and then update the ESKF with this pose. In the tightly coupled KN-LIO, starting from the nominal state, we iteratively update the Kalman filter by computing the SDF residuals of sampled points, until the ESKF converges. Finally, based on the optimized pose, we jointly optimize and update the local neural map by minimizing the BCE loss between sampled points and the neural field, and generate a dense mesh from the estimated neural field resorting to marching cubes.

B. Notation

We define the system state as:

$$\mathbf{x} = \{\mathbf{p}, \mathbf{v}, \mathbf{R}, \mathbf{b}_g, \mathbf{b}_a, \mathbf{g}\} \in \mathbb{M} = \mathbb{SO}(3) \times \mathbb{R}^{15}, \quad (1)$$

where the system's position ($\mathbf{p}$), velocity ($\mathbf{v}$), gyroscope bias ($\mathbf{b}_g$), accelerometer bias ($\mathbf{b}_a$), and gravity $\mathbf{g}$ are in $\mathbb{R}^3$, while the rotation ($\mathbf{R}$) is in $\mathbb{SO}(3)$. $\mathbf{p}$, $\mathbf{v}$, $\mathbf{R}$, and $\mathbf{g}$ are in the world frame by default, while $\mathbf{b}_g$ and $\mathbf{b}_a$ are in the body frame (IMU frame).

We define the corresponding error state to be estimated as:

$$\delta \mathbf{x} = [\delta \mathbf{p}^T, \delta \mathbf{v}^T, \delta \boldsymbol{\theta}^T, \delta \mathbf{b}_g^T, \delta \mathbf{b}_a^T, \delta \mathbf{g}^T] \in \mathbb{R}^{18}, \quad (2)$$

where $\delta \boldsymbol{\theta}$ is the axis-angle of the rotation error state $\delta \mathbf{R}$, which can be transformed from/to $\delta \mathbf{R}$ by logarithmic/exponential mapping.

C. Error-State Kalman Filter

The error-state Kalman filter (ESKF) considers the true state of the system as the sum of the nominal state and the error state. It recursively estimates the nominal state, error state, and error state covariance using the system inputs, and corrects the error state through external sensor observations to update the system state. Firstly, we briefly review the key concepts of ESKF. For a more detailed discussion, we recommend readers refer to Joan's [21] and Gao's work [43].

1) *Continuous-time Kinematics Model*: In continuous time, the motion of a rigid body system in an inertial frame follows Newton's laws of motion. The nominal kinematic model is:

$$\begin{aligned} \dot{\mathbf{p}} &= \mathbf{v}, \\ \dot{\mathbf{v}} &= -\mathbf{R}(\bar{\mathbf{a}} - \mathbf{b}_a - \boldsymbol{\eta}_a) + \mathbf{g}, \\ \dot{\mathbf{R}} &= \mathbf{R}(\tilde{\boldsymbol{\omega}} - \mathbf{b}_g - \boldsymbol{\eta}_g)^\wedge, \\ \dot{\mathbf{b}}_g &= \boldsymbol{\eta}_{bg}, \\ \dot{\mathbf{b}}_a &= \boldsymbol{\eta}_{ba}, \\ \dot{\mathbf{g}} &= \mathbf{0}, \end{aligned} \quad (3)$$

where $\tilde{a}$ and $\tilde{\omega}$ denote the IMU readings of acceleration and angular velocity, η_{ba} (η_{bg}) is the random impulse of accelerometer (gyroscope), $(\cdot)^\wedge$ means the skew-symmetric matrix of the given vector. The corresponding error-state kinematics model is formulated as:

$$\begin{aligned}\delta\dot{\mathbf{p}} &= \delta\mathbf{v}, \\ \delta\dot{\mathbf{v}} &= -\mathbf{R}(\tilde{\mathbf{a}} - \mathbf{b}_a)^\wedge \delta\boldsymbol{\theta} - \mathbf{R}\delta\mathbf{b}_a - \boldsymbol{\eta}_a + \delta\mathbf{g}, \\ \delta\dot{\boldsymbol{\theta}} &= -(\tilde{\boldsymbol{\omega}} - \mathbf{b}_g)^\wedge \delta\boldsymbol{\theta} - \delta\mathbf{b}_g - \boldsymbol{\eta}_g, \\ \delta\dot{\mathbf{b}}_g &= \boldsymbol{\eta}_{bg}, \\ \delta\dot{\mathbf{b}}_a &= \boldsymbol{\eta}_{ba}, \\ \delta\dot{\mathbf{g}} &= \mathbf{0}.\end{aligned}\quad (4)$$

2) *Discrete-time Kinematics Model*: By performing kinematic integration on the time dimension of the above equation, the nominal state kinematic equation in discrete time can be obtained as follows:

$$\begin{aligned}\mathbf{p}(t + \Delta t) &= \mathbf{p}(t) + \mathbf{v}\Delta t + \frac{1}{2}(\mathbf{R}(\tilde{\mathbf{a}} - \mathbf{b}_a)\Delta t^2 + \frac{1}{2}\mathbf{g}\Delta t^2, \\ \mathbf{v}(t + \Delta t) &= \mathbf{v}(t) + \mathbf{R}(\tilde{\mathbf{a}} - \mathbf{b}_a)\Delta t + \mathbf{g}\Delta t, \\ \mathbf{R}(t + \Delta t) &= \mathbf{R}(t)\text{Exp}((\tilde{\boldsymbol{\omega}} - \mathbf{b}_g)\Delta t), \\ \mathbf{b}_g(t + \Delta t) &= \mathbf{b}_g(t), \\ \mathbf{b}_a(t + \Delta t) &= \mathbf{b}_a(t), \\ \mathbf{g}(t + \Delta t) &= \mathbf{g}(t),\end{aligned}\quad (5)$$

where Δt is the integration interval and $\text{Exp}(\cdot)$ denotes the compound exponential map from $\mathbb{R}^3$ to $\text{SO}(3)$. Correspondingly, the error state kinematic equation for discrete-time is:

$$\begin{aligned}\delta\mathbf{p}(t + \Delta t) &= \delta\mathbf{p} + \delta\mathbf{v}\Delta t, \\ \delta\mathbf{v}(t + \Delta t) &= \delta\mathbf{v}(t) + (-\mathbf{R}(\tilde{\mathbf{a}} - \mathbf{b}_a)^\wedge \delta\boldsymbol{\theta} - \mathbf{R}\delta\mathbf{b}_a + \delta\mathbf{g})\Delta t - \boldsymbol{\eta}_v, \\ \delta\mathbf{R}(t + \Delta t) &= \mathbf{R}(t)\text{Exp}((\tilde{\boldsymbol{\omega}} - \mathbf{b}_g)\Delta t), \\ \delta\mathbf{b}_g(t + \Delta t) &= \delta\mathbf{b}_g + \boldsymbol{\eta}_g, \\ \delta\mathbf{b}_a(t + \Delta t) &= \delta\mathbf{b}_a + \boldsymbol{\eta}_a, \\ \delta\mathbf{g}(t + \Delta t) &= \delta\mathbf{g}.\end{aligned}\quad (6)$$

3) *Propagation, Update, and Reset*: According to the traditional form of the extended Kalman filter, the error state transition equation in matrix form can be obtained by reorganizing the above equations:

$$\begin{aligned}\delta\mathbf{x} &\leftarrow \mathbf{F}\delta\mathbf{x} + \boldsymbol{\omega}, \boldsymbol{\omega} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}), \\ \mathbf{Q} &= \Lambda(\mathbf{O}, \mathbf{Q}_{\eta_v}, \mathbf{Q}_{\eta_\theta}, \mathbf{Q}_{\eta_g}, \mathbf{Q}_{\eta_a}, \mathbf{O}),\end{aligned}\quad (7)$$

where $\mathbf{F}$ is the transition matrix of the error state which can be obtained from Eq. 6, $\mathbf{Q}$ is the covariance matrix of the error-state noise $\boldsymbol{\omega}$, and Λ denotes the diagonal matrix. For each step of error state propagation, the covariance of the error state ($\mathbf{P}$) needs to be mapped to a new tangent space by the transition matrix $\mathbf{F}$ and incorporate new noise:

$$\mathbf{P} \leftarrow \mathbf{F}\mathbf{P}\mathbf{F}^T + \mathbf{Q}.\quad (9)$$

If there are no new observations except the IMU readings, the uncertainty of the system will continue to increase with the recurrence of covariance. When a new sensor observation incomes, we can reduce this uncertainty by correcting the error

state. The update at this time can be carried out according to the standard form of the extended Kalman filter:

$$\mathbf{K} = \mathbf{P}\mathbf{H}^T(\mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{V})^{-1},\quad (10)$$

$$\delta\mathbf{x} = \mathbf{K}(\mathbf{z} - \mathbf{h}(\mathbf{x})),\quad (11)$$

$$\mathbf{x} \leftarrow \mathbf{x} + \delta\mathbf{x},\quad (12)$$

$$\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P},\quad (13)$$

where $\mathbf{V}$ is the covariance of the new measurements; $\mathbf{I}$ denotes an identity matrix; and $\mathbf{H}$ is the Jacobian matrix of the measurement $\mathbf{h}(\cdot)$ with respect to the error-state $\delta\mathbf{x}$, which can be obtained by the chain rule:

$$\mathbf{H} = \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \delta \mathbf{x}},\quad (14)$$

where the first term depends on the specific observation equation which will be discussed in subsequent sections, while the second term is known:

$$\frac{\partial \mathbf{x}}{\partial \delta \mathbf{x}} = \Lambda(\mathbf{I}_{3 \times 3}, \mathbf{I}_{3 \times 3}, \mathbf{J}_r^{-1}(\mathbf{R}), \mathbf{I}_{3 \times 3}, \mathbf{I}_{3 \times 3}, \mathbf{I}_{3 \times 3}),\quad (15)$$

$$\mathbf{J}_r^{-1}(\mathbf{R}) = \mathbf{I}_{3 \times 3} + \frac{1}{2}\boldsymbol{\theta}^\wedge + \left(\frac{1}{\|\boldsymbol{\theta}\|^2} - \frac{1 + \cos \|\boldsymbol{\theta}\|}{2\|\boldsymbol{\theta}\| \sin \|\boldsymbol{\theta}\|}\right)\boldsymbol{\theta}^\wedge^2,\quad (16)$$

where we explicitly marked the identity matrix shape with subscripts for better understanding.

After finishing an update of the error state and covariance, we need to reset them to prepare for the next update. To achieve this, the reset of the error state is trivial, resetting all variables to the zero vector is sufficient, while the covariance needs to be reset to the new state reference point. That is, the following reset operation is performed:

$$\delta\mathbf{x} = \mathbf{0},\quad (17)$$

$$\mathbf{P} = \mathbf{J}\mathbf{P}\mathbf{J}^T,\quad (18)$$

$$\mathbf{J} = \Lambda(\mathbf{I}_{6 \times 6}, \mathbf{J}_\theta, \mathbf{I}_{9 \times 9}),\quad (19)$$

$$\mathbf{J}_\theta = \mathbf{I}_{3 \times 3} - \frac{1}{2}\delta\boldsymbol{\theta}^\wedge.\quad (20)$$

D. System Initialization

Like most LiDAR-Inertial odometry systems, we initialize the system in a stationary state. We define the robot's starting point as the origin of the world frame, with the world frame's z -axis perpendicular to the gravity level, and the direction of gravity aligned with the negative z -axis. With this definition, we can accumulate multiple frames of IMU readings to estimate the initial orientation of the system. Let the accelerometer and gyroscope readings ($\mathcal{A}, \mathcal{W}$) during the initialization phase be:

$$\mathcal{A} = \{\tilde{\mathbf{a}}_0, \dots, \tilde{\mathbf{a}}_n\},\quad (21)$$

$$\mathcal{W} = \{\tilde{\boldsymbol{\omega}}_0, \dots, \tilde{\boldsymbol{\omega}}_n\}.\quad (22)$$

We believe that the bias of the accelerometer is a very small amount relative to the measured acceleration value, so the initial orientation of the carrier can be obtained by aligning the direction of the acceleration mean vector with the opposite direction of gravity:

$$\mathbf{R}_0 = \mathbf{R}(\mathbf{e}_{\overline{\mathcal{A}}}, \mathbf{e}_g),\quad (23)$$

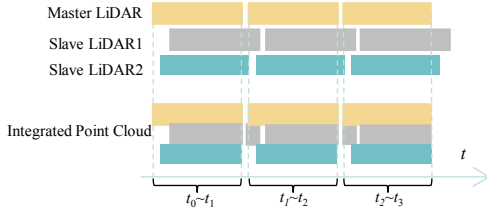


Fig. 2: Asynchronous LiDAR integration.

where e is a unit vector and $\bar{\mathcal{A}}$ is the mean of $\mathcal{A}$.

In static initialization, the common practice is to set the accelerometer bias to the zero vector. This has a minimal impact on LIO systems directly constrained by geometric lines or planes. But in neural field maps, where observations in the stationary state are limited and the map has not been fully learned, there may be transient drift phenomena. To solve this problem, on the basis of obtaining the initial rotation, if the scale of local gravity is known, we can also estimate the initial bias of the accelerometer:

$$\mathbf{b}_{a0} = \bar{\mathcal{A}} + \mathbf{R}_0^T \mathbf{g}. \quad (24)$$

As usual, the initial position and velocity are set as zero vectors, while the mean gyroscope reading is regarded as the initial guess of $\mathbf{b}_g$:

$$\mathbf{p}_0 = \mathbf{0}_3, \quad (25)$$

$$\mathbf{v}_0 = \mathbf{0}_3, \quad (26)$$

$$\mathbf{b}_{g0} = \bar{\mathcal{W}}. \quad (27)$$

E. Sensor Data Preprocessing

The rotating mechanical LiDAR has low point cloud resolution in the vertical direction, while the existing solid-state LiDAR can only present point clouds within the forward view. To enhance the robustness of LIO systems in various complex environments, integrating multi-view inputs is a more practical solution. However, this is not straightforward for feature-based approaches. Thanks to the direct registration of point clouds to neural maps, our system can be easily extended to accommodate multi-LiDAR inputs from complementary viewpoints. Specifically, for the input of multiple LiDAR point clouds (homogeneous or heterogeneous), we use the sampling time intervals of the master LiDAR as a reference and transform all points sampled by slave LiDARs within the same time interval to the coordinate system of the master LiDAR based on the calibrated extrinsics, generating a new integrated point cloud together with the master LiDAR's point cloud. Fig. 2 illustrates this operation.

For the high-frequency IMU data input, we recursively update the nominal state, error state, and covariance of the system as shown in Eq. 7 to Eq. 9. Subsequently, with the predicted relative transformation, we correct the distortion of the integrated point cloud by interpolation measuring poses between the starting and ending timestamps of the point cloud.

F. Neural Mapping

Currently, in neural SDF map representations for LiDAR, there are typically two organizational forms: octree-based and

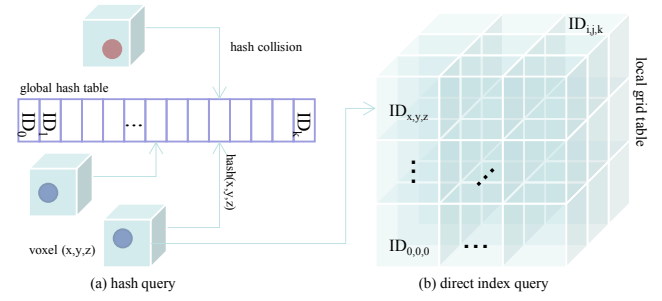


Fig. 3: Direct index query vs hash query.

point-based. The former divides space into fixed-size voxels at a certain resolution, storing voxels hit by point clouds in an octree, with each vertex of the voxel attached with a feature for learning scene representation. Although this method significantly reduces storage consumption, ray casting based on octrees remains a major efficiency bottleneck. The latter represents space as a series of neural points, with each point having learnable features attached, and efficient queries can be achieved by organizing these points using a KD-tree. Therefore, inspired by the approach of Pan *et al.* [16], this paper adopts the latter map representation form. In this way, the features of each point in space are characterized by a weighted representation based on its neighboring points, and a simple MLP decoder is leveraged to predict the closest distance to the scene surface from these features.

1) *Neural Point-based Signed Distance Field*: We implicitly divide the space into a regular grid with a certain resolution, where if a grid is observed by the actual point cloud, an actual neural point ($\mathbf{p}_n$) is assigned to it. This neural point is attached with an n-dimensional learnable vector ($\mathbf{v}$) to represent the features of the closest surface to the corresponding spatial position. The feature of any point in space is obtained by weighted averaging of its neighboring points based on inverse distance, and finally regressed to an SDF value by a simple MLP decoder:

$$\mathbf{s}_p = \mathcal{S}(\mathbf{p}) = \text{MLP}\left(\sum_{i=1}^n \frac{d_i}{\sum_{i=1}^n d_i} \mathbf{v}_i\right), \quad (28)$$

$$d_i = \frac{1}{\|\mathbf{p} - \mathbf{p}_{n_i}\|}, \quad (29)$$

as illustrated in “Neural Map Representation” of Fig. 1. The advantage of this explicit representation lies in two aspects: firstly, compared to implicit representations, it can accelerate the learning of map features; secondly, the mixed representation of a regular grid and sparse neural points is beneficial for designing fast query algorithms.

2) *Query with Direct Local Grid Indexing*: During online localization and mapping, it is necessary to query the SDF values of sampled points multiple times from the built neural field map to construct losses and iteratively optimize system state or map features. Therefore, efficient querying becomes a key factor in enhancing system efficiency. In traditional geometry-based LIO systems, nearest neighbor point searches are typically implemented using KD-Tree or iKD-Tree. However, in incremental mapping, Bai *et al.* [44] have demonstrated

that organizing the map based on sparse voxels and hash mapping is advantageous for leveraging parallel computing characteristics to achieve efficient retrieval. This approach has also been widely adopted in many neural mapping schemes. In experiments, we found that although this approach effectively balances space and time, two issues exist during querying. Firstly, each query requires first passing through a hash transformation to obtain a hash index; secondly, there is a risk of hash collisions where different spatial points may have the same hash value, requiring additional computational efforts to avoid this issue during each query. While the computational costs of these two aspects may not be significant in a single query, they would affect efficiency during the iterative process.

To avoid this issue, we provide a choice by pre-storing a regular search table of the local neural map in advance, allowing for direct retrieval of the index of the neural point corresponding to a spatial grid from its coordinates. Specifically, each time a new point cloud is added to the neural point map, we update the local map grid index table. Each grid in the table stores the index of the corresponding point in the global neural point cloud map. In this way, during querying, we can directly obtain the index of the corresponding neural point from the 3D grid coordinates of the query point. Compared to hash-based searching, this approach avoids the need for global hash table maintenance, hash mapping, and additional hash collision handling at the cost of the memory to maintain the local grid table.

3) *Incremental Update and Learning*: During mapping, given a point cloud with a known pose, our goal is to expand the existing neural map with new observations and leverage the point cloud to supervise learning the map features. To reconstruct a dense map, in addition to learning the surface features of the scene, we also need to model the free space. Therefore, using only surface points as supervision information is insufficient. Thus, we sample multiple points along the observation ray direction as samples to be learned. For a sampled point, the learning objective is to minimize the difference between the predicted SDF value and the actual observed SDF value. Following the suggestion of Zhong *et al.* [13], we first use the sigmoid function to scale the predicted SDF value ($s_{p_{s_i}}$) and target SDF values ($\hat{s}_{p_{s_i}}$) to (0, 1), and then calculate the binary cross-entropy loss of the SDF, i.e., the BCE loss:

$$\phi(s) = \frac{1}{1 + e^{s/\sigma}}, o_i = \phi(s_{p_{s_i}}), \hat{o}_i = \phi(\hat{s}_{p_{s_i}}), \quad (30)$$

$$L_{bce} = \frac{1}{N} \sum_{i=1}^N \hat{o}_i \log o_i + (1 - \hat{o}_i) \log(1 - o_i), \quad (31)$$

where σ is the scale factor. In addition, an ideal SDF field should satisfy the unit gradient modulus, i.e., the Eikonal equation, $\|\nabla S(\mathbf{p})\|_2 = 1$. Therefore, we also incorporate the Eikonal loss L_{eik} as a constraint:

$$L_{eik} = \frac{1}{N} \sum_{i=1}^N (\|\nabla S(\mathbf{p}_i)\|_2 - 1)^2. \quad (32)$$

When the sample point is close to the surface, it is acceptable to use the distance from that point to the endpoint ($\mathbf{p}_e$) as

the observed value of $s_{p_{s_i}}$. However, when the sample point is far from the surface or the angle of incidence is large, this approach can introduce significant errors. At this moment, the real SDF value should be the distance from the sampled point ($\mathbf{p}_s$) to the nearest point on the surface ($\mathbf{p}_n$) rather than from $\mathbf{p}_s$ to $\mathbf{p}_e$.

Fortunately, in the case of incremental mapping, we can obtain the neural field normal vector at the sampled point from the built neural field to obtain a corrected observed value. Estimating this normal vector not only helps correct the observed value, but we can also use the gradient information it provides in subsequent odometry estimation to register the point cloud to the map. In practical implementation, this gradient can be estimated numerically or obtained analytically from a differentiable computation graph.

G. Semi-Coupled Kinematic-Neural LiDAR-Inertial Odometry

In the semi-coupled mode, we consider treating point cloud and IMU observations relatively independently. That is, we first utilize an IMU readings to propagate the error-state Kalman filter and then correct the filter using the registration results. The advantage of this approach is that it decouples the data processing for the two types of sensors through result-level fusion, while simultaneously achieving timely correction of the system state and IMU biases by propagating external LiDAR observation information. By linking the chain of state propagation, point cloud de-skewing, registration with the neural map, and filter update, the system can achieve long-term stable running.

We perform direct registration from point clouds to neural maps to obtain LiDAR poses. Assuming the pose of point $\mathbf{p}$ is $\mathbf{T} = \{\mathbf{R}, \mathbf{t}\}$, the purpose of registration is to adjust the pose so that the SDF value at its spatial position approaches 0, that is:

$$\mathbf{R}^*, \mathbf{t}^* = \arg \min_{\mathbf{R}, \mathbf{t}} \sum_i \mathcal{S}(\mathbf{R}\mathbf{p}_i + \mathbf{t})^2. \quad (33)$$

To use traditional optimization algorithms such as the Levenberg-Marquardt algorithm to solve the optimal pose, we need to obtain the derivatives of $\mathcal{S}(\cdot)$ for rotation and translation parameters:

$$\frac{\partial \mathcal{S}(\mathbf{R}\mathbf{p}_i + \mathbf{t})}{\partial \mathbf{t}} = \frac{\partial \mathcal{S}(\mathbf{p}'_i)}{\partial \mathbf{p}'_i} = \mathbf{n}_i^T, \quad (34)$$

$$\frac{\partial \mathcal{S}(\mathbf{R}\mathbf{p}_i + \mathbf{t})}{\partial \boldsymbol{\theta}} = \frac{\partial \mathcal{S}(\mathbf{p}'_i)}{\partial \mathbf{p}'_i} \frac{\partial \mathbf{p}'_i}{\partial \boldsymbol{\theta}} = (\mathbf{p}'_i \times \mathbf{n}_i)^T, \quad (35)$$

$$\mathbf{J}_{\mathbf{t}, \boldsymbol{\theta}}^{f(\mathbf{p}_i)} = [\mathbf{n}_i^T, (\mathbf{p}'_i \times \mathbf{n}_i)^T], \quad (36)$$

where $\mathbf{n}_i$ denotes the gradient of the neural field at $\mathbf{p}'_i$ and $\boldsymbol{\theta}$ is the axis-angle of the rotation.

After obtaining the registered pose, updating the filter will become simpler, and the observation model at this point becomes:

$$\mathbf{z}_s = \mathbf{h}_s(\mathbf{x}) + \mathbf{v}_s = \mathbf{x} + \mathbf{v}_s, \quad \mathbf{v}_s \in \mathcal{N}(\mathbf{0}, \mathbf{V}_s), \quad (37)$$

where $\mathbf{v}_s$ denotes the noise of the point cloud to neural distance field registration results. Since the Jacobians of $\mathbf{h}_s(\mathbf{x})$ with respect to the nominal state at this time is the identity

matrix, the update matrix in Eq. 10 and Eq. 13 becomes the same as that of Eq. 15.

H. Tightly Coupled Kinematic-Neural LiDAR-Inertial Odometry

Although the semi-coupled mode can leverage LiDAR registration results to correct IMU biases and offers the advantage of relatively decoupling heterogeneous observations, the approach of first registering and then optimizing biases may entail certain risks of information loss. Therefore, this paper also tightly couples the registration of point clouds to the neural field with the estimation of IMU biases. With the goal of finding the optimal registration, we combine the estimation of the error-state Kalman filter with the iterative registration process, known as the iterative Kalman filter.

In each iteration, we solve the following least squares problem:

$$\delta \mathbf{x}^* = \arg \min_{\delta \mathbf{x}} \|\mathbf{z} - \mathbf{H}(\mathbf{x} \oplus \delta \mathbf{x})\|_{\mathbf{V}}^2 + \|\delta \mathbf{x}\|_{\mathbf{P}}^2, \quad (38)$$

where $\oplus$ indicates the plus operation on the manifold $\mathbb{M}$. In the Bayesian framework, this objective can be derived to yield analytical solutions as shown in Eq. 10~Eq. 13. As shown in these equations, to perform tightly coupled iterations of the Kalman filter, we need to determine the observation equation at this time and the corresponding matrix $\mathbf{H}$. Our registration objective remains as shown in Eq. 33, thus the observation model can be defined as:

$$\begin{aligned} \mathbf{z}_t &= \mathbf{h}_t(\mathbf{p}|\mathbf{x}) + \mathbf{v}_t \\ &= \begin{bmatrix} h(\mathbf{p}_0|\mathbf{x}) \\ \vdots \\ h(\mathbf{p}_n|\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \mathcal{S}_0(\mathbf{R}\mathbf{p}_0 + \mathbf{t}) \\ \vdots \\ \mathcal{S}_n(\mathbf{R}\mathbf{p}_n + \mathbf{t}) \end{bmatrix}, \end{aligned} \quad (39)$$

$$\mathbf{v}_t \in \mathcal{N}(\mathbf{0}, \mathbf{V}_t), \quad (40)$$

To update the Kalman filter using this observation model, we need to calculate the partial derivative of $h(\cdot)$ with respect to $\mathbf{x}$:

$$\begin{aligned} \frac{\partial h(\mathbf{p}_i|\mathbf{x})}{\partial \mathbf{x}} &= \frac{\partial \mathcal{S}_i}{\partial \mathbf{p}'} \frac{\partial \mathbf{p}'}{\partial \mathbf{x}} \\ &= \mathbf{n}_i^T \begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{O}_{3 \times 3} & -\mathbf{R}\mathbf{p}_i^\wedge & \mathbf{O}_{3 \times 9} \end{bmatrix}, \end{aligned} \quad (41)$$

where $\mathbf{O}_{3 \times 3}/\mathbf{O}_{3 \times 9}$ is a $3 \times 3/3 \times 9$ zero matrix. Thus, the i -th row of the matrix $\mathbf{H}$ is determined by the chain rule combining Eq. 14 to Eq. 16:

$$\mathbf{H}_i = \frac{\partial \mathbf{h}_i}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \delta \mathbf{x}} = \begin{bmatrix} \mathbf{n}_i^T & -\mathbf{0}_3^T & \mathbf{n}_i^T \mathbf{R}\mathbf{p}_i^\wedge \mathbf{J}_r^{-1}(\mathbf{R}) & \mathbf{0}_9^T \end{bmatrix}, \quad (42)$$

where $\mathbf{0}_3/\mathbf{0}_9$ denotes a zero vector with 3/9 elements.

We can estimate the error state and update the covariance according to Eq. 10~Eq. 13, however, a single point cloud typically contains thousands of points, which would make the inversion part in Eq. 10 computationally expensive. To address this issue, we resort to the Sherman-Morrison-Woodbury identity transformation and calculate the matrix $\mathbf{K}$ in an equivalent way as follows:

$$\mathbf{K} = (\mathbf{P}^{-1} + \mathbf{H}^T \mathbf{V}^{-1} \mathbf{H})^{-1} \mathbf{H}^T \mathbf{V}^{-1}. \quad (43)$$

As a result, the dimension of the inversion in each iteration is reduced from a variable N (number of points) to a constant 18 (the dimension of error state), making the system more efficient and stable. In each iteration, we calculate the error state according to Eq. 43 and Eq. 11 and update the state by Eq. 13. If the value of the current error state is below a certain threshold or reaches the maximum number of iterations, we consider the iteration to be complete, and transform the covariance matrix to the position of the last iteration using Eq. 13.

To summarize, we reformat the pseudocodes of our Semi-KN-LIO and KN-LIO in Algorithm 1.

IV. EVALUATION SETUP

A. Dataset

We evaluated our KN-LIO and semi-KN-LIO on three public datasets and one self-collected dataset.

1) *VIRAL*: One of them is the VIRAL dataset collected from a drone, consisting of 18 sequences from various indoor and outdoor courtyard scenes, including both structured and unstructured environments. Among these, the first 9 sequences exhibit relatively smooth motion, while the subsequent 9 sequences involve more aggressive drone movements, such as rapid ascents and fast rotations. Most existing methods have only been tested on the former 9 sequences. In this paper, to conduct a more comprehensive evaluation, we experiment with relevant competing approaches on all sequences and report detailed results accordingly.

2) *HILTI2022*: The HILTI2022 dataset is collected from a construction site, where each sequence comprises featureless and varying lighting environments, posing challenges for an LIO system. This dataset is captured using a handheld device equipped with a Hesai PandarXT-32 LiDAR and Sevensense Alphasense Core. We evaluated the relevant algorithms on sequences with publicly available 6-DoF millimeter-level ground truth poses, specifically including “Exp04 Construction Upper Level 1” (Exp04), “Exp05 Construction Upper Level 2” (Exp05), “Exp06 Construction Upper Level 3” (Exp06), “Exp14 Basement 2” (Exp14), “Exp16 Attic to Upper Gallery 2” (Exp16), and “Exp18 Corridor Lower Gallery 2” (Exp18).

3) *Newer College*: We conducted quantitative testing of the algorithm for mapping on Newer College. The dataset provides scenes reconstructed using survey-grade laser scanners. Typically, pure LiDAR mapping methods are tested on segments extracted from the sequence “02_long_experiment”. For comparison purposes, we also extracted the corresponding IMU data from the raw dataset, while retaining IMU readings corresponding to 10 frames of point clouds for system initialization. It should be clarified that these 10 frames of point clouds within the corresponding time period were not involved in map reconstruction.

4) *HD-Mid*: We developed a handheld device for collecting test datasets. The device includes two DJI Mid360 LiDAR sensors, one facing upwards and one facing downwards, each equipped with a consumer-grade IMU. To comprehensively evaluate the algorithm, we collected various complex scenarios indoors and outdoors, including long corridors, multiple flights

Algorithm 1 Pseudocodes of Semi-KN-LIO and KN-LIO

Input: Point cloud ($\mathcal{P}$), IMU accelerometer and gyroscope readings ($\mathcal{A}$ and $\mathcal{W}$).
Output: System state ($\mathbf{x}$) and updated neural SDF field ($\mathcal{S}$).
1: **if** not initialized **then**
2: Estimate the initial rotation $\mathbf{R}_0$ via Eq. 23.
3: Estimate the initial accelerometer bias via Eq. 24.
4: Estimate the initial gyroscope bias via Eq. 27.
5: Set the initial position ($\mathbf{p}_0$) and the initial velocity ($\mathbf{v}_0$) as zero vectors.
6: Voxel sampling neural points $\mathcal{N}_0$ from $\mathcal{P}$.
7: Sampling map points $\mathcal{M}_0$ along the rays of $\mathcal{N}_0$.
8: Learning the neural points features of $\mathcal{S}$ via Eq. 29, Eq. 31, and Eq. 32.
9: Set system initialized.
10: **else**
11: Propagate the system's nominal state via Eq. 5.
12: Propagate the system error state via Eq. 7.
13: Propagate the system's covariance via Eq. 9.
14: De-skew the point cloud using the predicted system state and timestamps, resulting in $\tilde{\mathcal{P}}$.
15: **if** Semi-KN-LIO **then**
16: Register the de-skewed point cloud to the built map via Eq. 33.
17: Combine Eq. 37, Eq. 15, Eq. 16, and Eq. 14 to get the observation matrix.
18: Update the system error state via Eq. 10 ~ Eq. 13.
19: Reset the error state using Eq. 17 ~ Eq. 20.
20: **else if** KN-LIO **then**
21: **while** the Kalman filter is not converged **do**
22: Determine the i -th row of the observation matrix according to Eq. 42 using the i -th de-skewed point.
23: Get the Kalman gain according to Eq. 43.
24: Update the system state via Eq. 10 ~ Eq. 13.
25: Reset the error state using Eq. 17 ~ Eq. 20.
26: **if** converged **then**
27: break.
28: **end if**
29: **end while**
30: **end if**
31: Voxel sampling neural points $\mathcal{N}_i$ from $\tilde{\mathcal{P}}$.
32: Sampling map points $\mathcal{M}_i$ along the rays of $\mathcal{N}_i$.
33: Joint learning the neural point features of $\mathcal{S}$ and updating the LiDAR pose via Eq. 29, Eq. 31, and Eq. 32.
34: **end if**

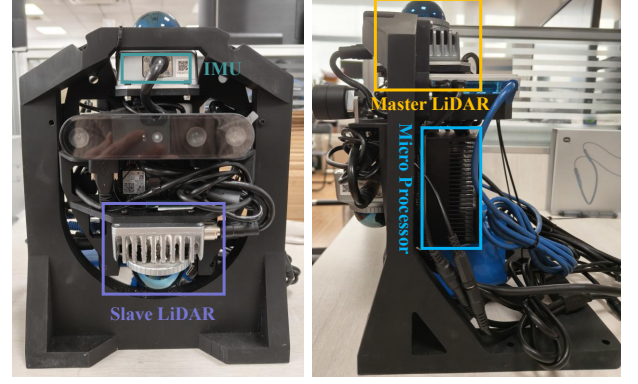


Fig. 4: Self-developed handheld device.

of a method, we utilized the Umeyama algorithm [45] to estimate its transformation relative to the ground truth, and subsequently calculated its Root Mean Square Error (RMSE).

In terms of mapping, following the method proposed in [46], we assessed the Accuracy (Acc.), Completeness (Comp.), $l1$ Chamfer distance ($C-l1$), and F-Score of the reconstructed maps generated by the algorithm. The specific calculation methods for these indicators are as follows.

Let the reconstructed map be $\mathcal{M}_r$ and the ground truth map be $\mathcal{M}_g$. A point in $\mathcal{M}_r/\mathcal{M}_g$ is denoted by $\mathbf{r}/\mathbf{g}$, respectively. Thus, we can define the error of a reconstructed point to the ground truth map ($e_{\mathbf{r} \rightarrow \mathcal{M}_g}$) and the error of a ground truth point to the reconstructed map ($e_{\mathbf{g} \rightarrow \mathcal{M}_r}$) by:

$$e_{\mathbf{r} \rightarrow \mathcal{M}_g} = \min_{\mathbf{g} \in \mathcal{M}_g} \|\mathbf{r} - \mathbf{g}\|, \quad (44)$$

$$e_{\mathbf{g} \rightarrow \mathcal{M}_r} = \min_{\mathbf{r} \in \mathcal{M}_r} \|\mathbf{g} - \mathbf{r}\|. \quad (45)$$

Accuracy. Mapping accuracy refers to the average error between reconstructed points and their corresponding points on the ground truth map:

$$Accuracy = \frac{1}{|\mathcal{M}_r|} \sum_{\mathbf{r} \in \mathcal{M}_r} e_{\mathbf{r} \rightarrow \mathcal{M}_g}. \quad (46)$$

Completeness. Mapping completeness refers to the average error between ground truth map points and their corresponding reconstructed points:

$$Completeness = \frac{1}{|\mathcal{M}_g|} \sum_{\mathbf{g} \in \mathcal{M}_g} e_{\mathbf{g} \rightarrow \mathcal{M}_r}. \quad (47)$$

Chamfer distance. As typical mapping schemes do [13], [15], we also evaluate the $l1$ Chamfer distance (C_{l1}):

$$C_{l1} = 0.5 * (Accuracy + Completeness). \quad (48)$$

F-score. Let d be the acceptance distance threshold of $e_{\mathbf{r} \rightarrow \mathcal{M}_g}$ and $e_{\mathbf{g} \rightarrow \mathcal{M}_r}$. The mapping *precision* (PR) and *recall* (RC) can be obtained via:

$$PR = \frac{1}{|\mathcal{M}_r|} \sum_{\mathbf{r} \in \mathcal{M}_r} (e_{\mathbf{r} \rightarrow \mathcal{M}_g} < d), \quad (49)$$

$$RC = \frac{1}{|\mathcal{M}_g|} \sum_{\mathbf{g} \in \mathcal{M}_g} (e_{\mathbf{g} \rightarrow \mathcal{M}_r} < d), \quad (50)$$

of stairs, two-story buildings, and forest undergrowth. For each collected sequence, the start and end points of the trajectory coincide to assess the drift magnitude of the odometry algorithm through absolute position differences. The device structure is shown in Fig. 4.

B. Metrics

Regarding localization, following the common practice in related methods [6], [16], we evaluated the absolute positioning error of the approaches. For each estimated pose sequence

More compactly, precision and recall can be expressed as a single metric:

$$F\text{-score} = \frac{2 * PR * RC}{PR + RC}. \quad (51)$$

According to these definitions, it can be inferred that lower values for Accuracy, Completeness, and Chamfer distance indicate better reconstruction quality, while a higher value for F-score signifies a higher recall rate and precision.

C. Main Comparing Methods

1) LiDAR/LiDAR-Inertial Odometry:

- PIN-SLAM [16]. It first introduces neural point representation into LiDAR SLAM, achieving efficient tracking and online mapping.
- SLAMesh [47]. It is based on geometric mesh representation to construct constraints between points and mesh faces, thereby optimizing the pose of the carrier.
- D-LIOM [4]. Based on a probabilistic occupancy map representation, D-LIOM achieves coupled pose estimation of direct registration and IMU.
- Ct-LIO [26]. This method is based on continuous-time trajectory representation and conducts step-by-step point cloud registration and trajectory optimization.
- dllo [5]. Based on GICP, this method achieves robust localization and mapping through alternating point cloud registration and IMU correction.
- LIO-SAM [3]. Building upon LOAM point cloud features, this method corrects the high-frequency front-end IMU preintegration recursive pose with low-frequency, high-precision point cloud-to-map registration results.
- Fast-LIO2 [6]. Based on the error-state Kalman filter, this method tightly couples system state optimization by minimizing the distance between points and their nearest neighbor plane elements.

2) Dense Mapping:

- SLAMesh [47]. Based on the online estimated pose, SLAMesh also incrementally generates dense geometric mesh online.
- ImMesh [34]. Building upon LiDAR-inertial pose estimation, ImMesh proposes an efficient method to generate meshes online from the already localized point cloud.
- SHINE-Mapping [13]. It utilizes octree to organize large-scale scenes, updating the neural field by learning features attached to octree leaf nodes.
- NeRF-LOAM [15]. It represents sparse scenes as octrees with attached neural features, optimizing the pose and neural field by minimizing the SDF residual of the ray sampled points.
- PIN-SLAM [16]. Under the tracked pose results, PIN-SLAM jointly optimizes keyframe poses and neural point features within the local map area.

As Table I shows, in terms of map representation, apart from our KN-LIO and semi-KN-LIO, only PIN-SLAM and SLAMesh support incremental dense reconstruction, with only PIN-SLAM and our approach being based on neural field representation, enabling continuous Signed Distance Function (SDF) field estimation. Regarding pose estimation, while

TABLE I: Traits of LiDAR/LiDAR-Inertial Odometry systems.

Method	Dense	Direct	Inertial	Neural	Multi-LiDAR
PIN-SLAM	✓	✓	✗	✓	✗
SLAMesh	✓	✓	✗	✗	✗
D-LIOM	✗	✓	✓	✗	✓
Ct-LIO	✗	✓	✓	✗	✓
dllo	✗	✓	✓	✗	✗
LIO-SAM	✗	✗	✓	✗	✗
Fast-LIO2	✗	✓	✓	✗	✗
Semi-KN-LIO	✓	✓	✓	✓	✓
KN-LIO	✓	✓	✓	✓	✓

several methods are feature-free, except for our approach, only D-LIOM and Ct-LIO support multiple LiDAR inputs. Furthermore, among all methods, our approach simultaneously supports dense reconstruction, direct registration, inertial information fusion, neural representation, and multi-LiDAR information fusion.

V. EXPERIMENTS

In this section, we conduct comprehensive experiments to demonstrate the superior qualities of our KN-LIO system:

- 1) Comparable pose tracking accuracy to the state-of-the-art LiDAR-Inertial odometry.
- 2) Better mapping performance compared to existing dense mapping approaches.
- 3) Strong robustness to handle rapid motion and challenging scenarios.
- 4) Good scalability to support multi-lidar inputs with asynchronous timestamps.

A. Results of Localization

We tested the localization performance of the algorithm on the representative datasets VIRAL and HILTI2022, respectively. For specific methods, we utilized the corresponding open-source code provided in their papers for testing. We followed the parameter settings recommended in the original papers. As some of the code did not support these datasets, we made adjustments such as sensor type and parameter adaptation. During testing, some algorithms did not yield meaningful results or diverged in certain sequences, which we uniformly labeled as tracking failures. The pose tracking results on the VIRAL and HILTI2022 datasets are recorded in Table II.

1) *Tracking Accuracy:* From Table II, first analysing the results on VIRAL, it is evident that the successful integration of the IMU in our approach enables KN-LIO to achieve higher accuracy and better robustness compared to pure LiDAR-based dense SLAM methods (such as the neural field-based PIN-SLAM and the geometric mesh-based SLAMesh). Our KN-LIO outperforms the excellent-performing LIO-SAM in geometric feature-based methods. Compared to traditional direct approaches (D-LIOM, Ct-LIO, and dllo), our neural field-based pose estimation achieves higher accuracy, whether in semi-coupled or tightly coupled way. Compared to the state-of-the-art direct method Fast-LIO2, our performance is still comparable, with Fast-LIO2 achieving the best results on 7 sequences and our approach achieving the best on 8

TABLE II: RMSEs (m) of advanced LiDAR/LiDAR-Inertial Odometry systems on VIRAL and HILTI22. $\times$ means failure. ‘-’ implies the result is not available.

	PIN-SLAM [16]	SLAMesh [47]	D-LIOM [4]	Ct-LIO [26]	dllo [5]	LIO-SAM [3]	Fast-LIO2 [6]	Semi-KN-LIO	KN-LIO
eee_01	$\times$	0.089	0.191	0.186	0.143	0.074	0.068	0.112	<u>0.072</u>
eee_02	2.056	0.081	0.508	0.416	0.141	0.067	0.075	0.092	<u>0.071</u>
eee_03	0.615	0.122	0.220	0.326	0.186	0.117	0.109	<u>0.109</u>	0.087
nya_01	0.086	0.085	0.154	0.160	0.110	0.077	0.060	0.086	<u>0.075</u>
nya_02	0.098	0.095	0.200	0.159	0.154	0.091	0.095	<u>0.093</u>	0.083
nya_03	0.455	0.086	0.202	0.203	0.175	0.147	0.103	<u>0.094</u>	0.083
sbs_01	0.205	0.097	0.178	0.165	0.146	0.088	<u>0.084</u>	0.097	0.083
sbs_02	0.528	0.097	0.165	0.176	0.133	<u>0.089</u>	0.076	0.094	0.091
sbs_03	0.708	0.100	0.154	0.375	0.146	0.083	0.076	0.095	<u>0.080</u>
rtp_01	$\times$	0.259	2.163	0.240	0.371	0.273	0.125	0.326	<u>0.129</u>
rtp_02	0.143	0.429	0.567	0.516	0.605	0.154	<u>0.131</u>	0.137	0.111
rtp_03	$\times$	0.146	$\times$	0.223	0.284	0.106	0.137	0.143	<u>0.121</u>
spms_01	$\times$	1.125	$\times$	$\times$	1.426	0.626	<u>0.210</u>	$\times$	0.162
spms_02	$\times$	$\times$	$\times$	$\times$	1.698	$\times$	<u>0.336</u>	$\times$	0.275
spms_03	$\times$	$\times$	$\times$	1.005	0.732	$\times$	0.175	$\times$	<u>0.192</u>
tnp_01	2.969	0.320	0.879	1.937	0.159	0.081	0.090	0.076	<u>0.081</u>
tnp_02	0.653	0.292	1.518	$\times$	0.139	0.138	0.109	0.079	0.074
tnp_03	0.887	0.163	1.179	$\times$	0.141	$\times$	0.089	0.089	<u>0.101</u>
exp04	0.080	$\times$	$\times$	$\times$	$\times$	-	0.022	0.098	<u>0.069</u>
exp05	0.070	$\times$	$\times$	$\times$	$\times$	-	0.020	0.081	<u>0.067</u>
exp06	$\times$	$\times$	$\times$	$\times$	$\times$	-	0.037	0.190	<u>0.133</u>
exp14	0.120	$\times$	$\times$	$\times$	$\times$	-	0.070	0.888	<u>0.092</u>
exp16	$\times$	$\times$	$\times$	$\times$	$\times$	-	<u>1.129</u>	$\times$	0.912
exp18	$\times$	$\times$	$\times$	$\times$	$\times$	-	0.231	$\times$	<u>0.236</u>

sequences. When tested under the HILTI22 scenario, although our positioning accuracy is not as good as the state-of-the-art Fast-LIO2 on the relatively simple exp04, exp05, and exp06 sequences, on the more complex sequences exp14, exp16, and exp18, our accuracy is comparable to Fast-LIO2, with no significant difference.

2) *Tracking Robustness*: The learning-based PIN-SLAM failed in 6 sequences in the VIRAL dataset and 3 sequences in HILTI2022. In comparison, our semi-coupled approach had fewer failures, while the tightly coupled one successfully ran all sequences. This result demonstrates that effective fusion of IMU data can enhance system robustness. Compared to traditional geometry-based LiDAR-Inertial odometry systems, only dllo, Fast-LIO2, and our KN-LIO achieve successful localization in all sequences on the VIRAL dataset; on HILTI2022, only Fast-LIO2 and KN-LIO succeed in localization. This indicates that our coupled approach combining geometric kinematics and point cloud to neural field registration can achieve robustness comparable to traditional kinematic and geometric coupling methods, effectively handling intense vehicle motion and complex scenarios.

B. Results of Mapping

1) *Qualitative*: For the 6 scenes and 18 sequences in the VIRAL dataset, we conducted separate tests using PIN-SLAM, ImMesh, and our KN-LIO. We plotted the mapping results of each algorithm achieving the best pose tracking in each scene in Fig. 5. It can be observed that in terms of completeness, KN-LIO demonstrates the best capability, for example, in the grounds in “rtp” and “spms”. In terms of detailed reconstruction, KN-LIO also exhibited higher accuracy, such as in the trees in “eee”, the streetlights in “sbs”, and the spiral staircase in “tnp”. Furthermore, we found that the geometry-based ImMesh method tends to produce more noise compared to the neural field-based PIN-SLAM and KN-LIO, which is detrimental to detailed reconstruction.

TABLE III: Mapping stats on Newer College’s ‘quad’.

Method	Pose	Acc.	Comp.	C-L1	F-Score
VDB-Fusion [48]	KissICP [50]	14.03	25.46	19.75	69.50
SHINE [13]		14.87	20.02	17.45	68.85
NKSR [49]		15.67	36.87	26.67	58.57
PUMA [8]	Odometry	15.30	71.91	43.60	57.27
SLAMesh [47]		19.21	48.83	34.02	45.24
NeRF-LOAM [15]		12.89	22.21	17.55	74.37
ImMesh [34]		15.05	19.80	17.42	66.87
PIN-SLAM [16]		11.55	15.25	13.40	82.08
KN-LIO (ours)		8.18	11.65	9.92	91.01

2) *Quantitative*: In terms of quantitative mapping results, Table III presents the relevant results of different mapping algorithms at Newer College. VDB-Fusion [48], SHINE-Mapping [13], and NKSR [49] are pure mapping algorithms, with their poses provided by the state-of-the-art LiDAR Odometry method KissICP [50], while PUMA [8], SLAMesh [47], NeRF-LOAM [15], ImMesh [34], PIN-SLAM [16], and our KN-LIO estimate poses online within their algorithms. The data in Table III shows that our KN-LIO performed well across all metrics. Compared to the state-of-the-art LiDAR-based mapping method PIN-SLAM, KN-LIO significantly improves reconstruction accuracy and completeness through effective integration of the IMU. In comparison to ImMesh, which estimates poses based on Fast-LIO2, the superior mapping performance of KN-LIO indicates that the learning-based approach is more conducive to the effective fusion of multi-frame observation data, leading to noise reduction and accuracy improvement.

C. Multi-LiDAR Support

To validate (Semi-)KN-LIO’s support for multiple LiDAR inputs, we simultaneously utilized the horizontal and vertical LiDAR data from the VIRAL dataset to analyze their impact on the system’s localization and mapping performance. As

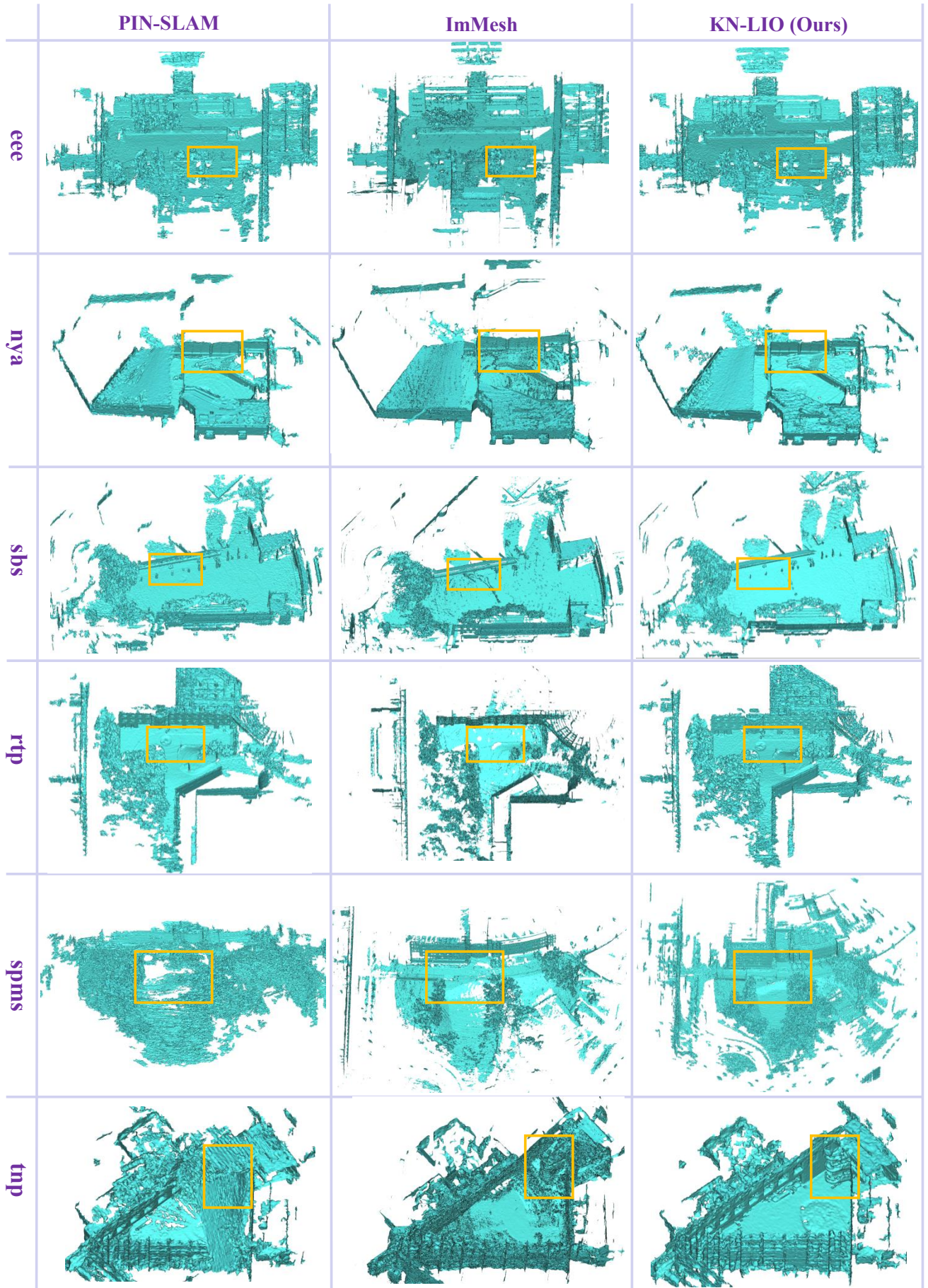


Fig. 5: Meshes reconstructed on VIRAL experimental sites by PIN-SLAM, ImMesh, and our KN-LIO.
 Preprint submitted to IEEE Transactions on Robotics. Received: March 3, 2025 18:53:58 Pacific Time

shown in Table IV, with the fusion of multi-view data, our approach exhibited higher pose estimation accuracy and better robustness compared to traditional probabilistic voxel-based methods. Compared to the results of single LiDAR scenarios in the table, it can be observed that the improvement in positioning accuracy due to complementary views is also significant. Furthermore, with dual LiDAR inputs, the advantages of tightly coupled fusion over semi-coupled integration are more pronounced. Fig. 6 illustrates the reconstructed map under single/dual LiDAR inputs. It is evident that KN-LIO effectively supports the input of multi-view LiDAR, resulting in a more comprehensive reconstructed environmental map.

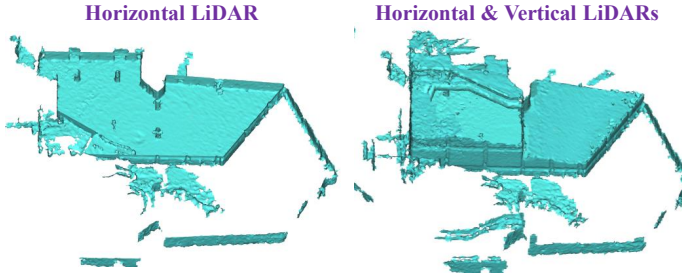


Fig. 6: Mapping with the horizontal LiDAR (left) and with both the horizontal and vertical LiDARs (right).

TABLE IV: RMSEs (m) on VIRAL with dual LiDAR inputs.

	D-LIOM-HV	Ct-LIO-HV	Semi-KN-LIO	KN-LIO
eee_01	0.124	0.132	0.095	0.072
eee_02	0.122	0.247	0.091	0.067
eee_03	0.150	0.137	0.099	0.087
nya_01	0.137	0.108	0.073	0.071
nya_02	0.161	0.179	0.090	0.077
nya_03	0.159	0.240	0.081	0.076
rtp_01	0.240	0.211	0.196	0.142
rtp_02	0.192	0.223	0.156	0.108
rtp_03	0.223	0.335	0.112	0.107
sbs_01	0.136	0.170	0.085	0.081
sbs_02	0.125	0.138	0.093	0.098
sbs_03	0.112	0.152	0.067	0.089
spms_01	x	x	x	0.143
spms_02	x	x	x	0.295
spms_03	1.005	1.015	x	0.238
tnp_01	0.130	0.178	0.077	0.076
tnp_02	0.143	0.168	0.058	0.059
tnp_03	0.134	0.170	0.088	0.099

D. Verification on Self-collected Dataset

In addition to the aforementioned public datasets, we further validate the efficacy of the proposed methods on self-collected datasets. Given the absence of ground-truth data, a qualitative assessment is conducted to evaluate the algorithm's performance on HD-Mid.

As illustrated in Fig. 7, regarding localization capability, our framework successfully returns to initial positions in both Forest and Stair sequences of the collected data, demonstrating its robustness in unstructured environments or with complex non-planar motions. For extended planar and cross-floor sequences, the drift magnitude of our approach remains comparable to Fast-LIO2 [6], a state-of-the-art geometry-constrained solution, consistent with quantitative findings in Sec. V-A.

In mapping performance comparison, our framework exhibits superior completeness and detail reconstruction accuracy relative to dense mapping baselines, as evidenced by the reconstruction quality within orange-bounded regions. When contrasted with neural reconstruction approaches like PIN-SLAM [16], the effective integration of IMU measurements enhances local reconstruction precision in our system. Compared with geometry-based reconstruction framework, ImMesh [34], our neural reconstruction paradigm significantly improves scene completeness. Furthermore, traditional geometry-centric LiDAR-Inertial odometry solution (Fast-LIO2) generates sparser point cloud maps that are less suitable for direct utilization in downstream applications compared to the dense ones.

E. Ablation Study

TABLE V: Time costs of query operations (ms).

t1	t2	t3	t4	t5	t6
0.519	0.136	20.902	24.514	13.370	10.278

1) *Semi-coupled vs Tightly-Coupled*: Comparing the results of Semi-KN-LIO and KN-LIO, although semi-coupling significantly improves positioning accuracy compared to methods purely using LiDAR data, its performance is not as good as tightly coupled methods in terms of accuracy and robustness, especially on challenging sequences. It can be seen that although tightly coupled methods are more complex, their lossless information fusion process can better leverage the complementary characteristics of sensors.

2) *Direct Indexing vs Hash Mapping*: To evaluate the efficiency difference between direct grid indexing and hash querying, we define the following time consumptions:

- t1: Average time cost for a single hash query.
- t2: Average time cost for a single grid index query.
- t3: Average time cost for hash queries in a complete tracking process.
- t4: Average time cost for grid index queries in a complete tracking process.
- t5: Average time cost for hash queries in a complete mapping process.
- t6: Average time cost for grid index queries in a complete mapping process.

We statistically analyzed the corresponding results and listed them in Table V. It can be observed that, compared to hash querying, direct grid indexing avoids hash mapping and the handling of hash collisions, thereby demonstrating a significant efficiency advantage in single queries (nearly 4 times improvement). When performing complete tracking and mapping processes, which involve multiple queries, grid indexing theoretically also possesses a higher efficiency advantage. However, the results show that it is more efficient in mapping but slower in tracking compared to hash querying. The reasons for this are twofold: firstly, parallel processing diminishes the advantage of single queries. Secondly, local queries reduce the number of points to be matched compared to global queries, potentially increasing the number of iterations in tracking.

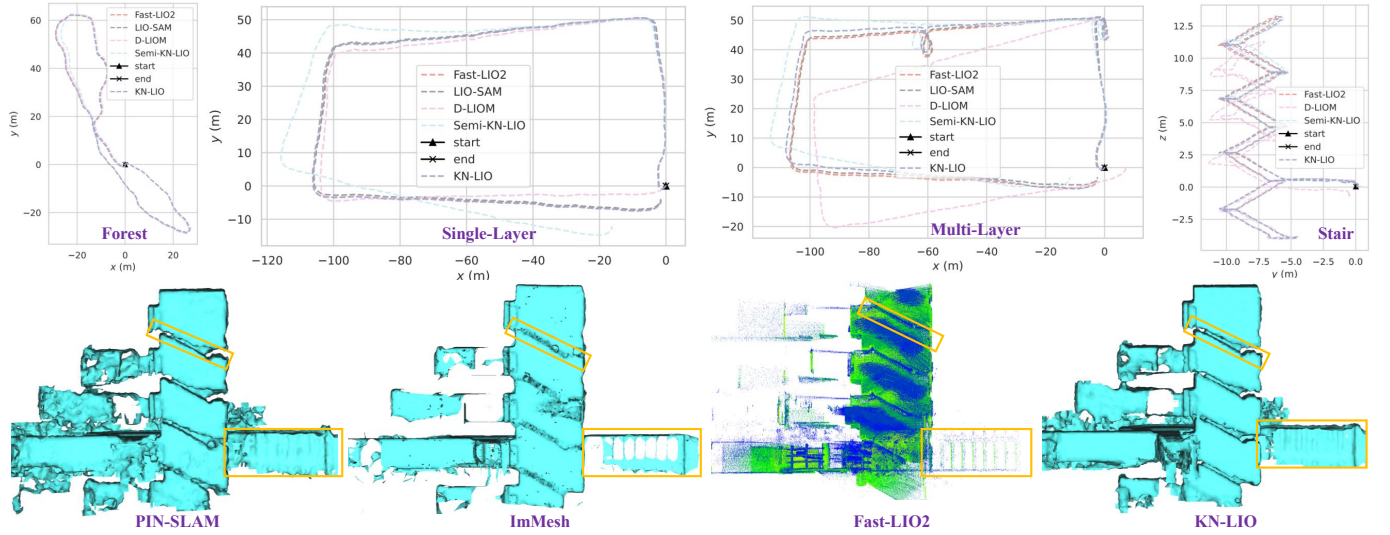


Fig. 7: Trajectories and maps of the HD-Mid dataset estimated by LiDAR-Inertial odometry and dense mapping systems.

During mapping, since only neural points within the local map are optimized, it can lead to higher efficiency, which may be more crucial for higher-quality dense mapping requirements.

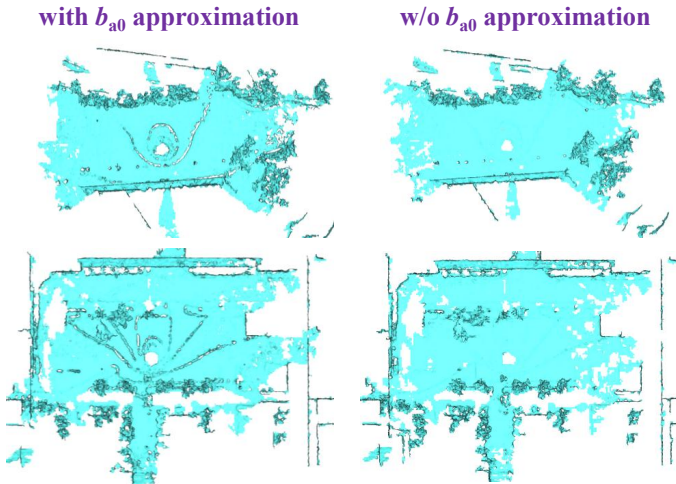


Fig. 8: Mapping with the horizontal LiDAR (left) and with both the horizontal and vertical LiDARs (right).

3) *Initial Accelerometer Bias Approximation*: We demonstrate the positive impact of initial approximation of accelerometer bias (b_{a0}) on sequences eee01 and sbs01. We tested two methods of compensating for b_{a0} : directly setting it as the zero vector and setting it according to Eq. 24. As shown in Fig. 8, when initial estimates are available, the mapping is noticeably more accurate. The fundamental reason is that limited observations in the stationary state make it difficult to fully learn the map, leading to the system state being dominated by accelerometer bias and causing drift. Therefore, with the assumption of local gravity acceleration magnitude, a reasonable initial estimate of b_{a0} is necessary.

F. Limitations and Potential Improvements

Although the effective fusion of geometric kinematics and neural fields makes KN-LIO competitive in both pose esti-

mation and dense mapping, there are still some limitations that need to be addressed. Firstly, despite the significant improvement in query and learning efficiency with neural point representation, the algorithm still relies on high computational power GPUs, making real-time computation challenging on embedded platforms. We believe that this goal could be achieved through more efficient implementation and design choices, such as using C++ instead of Python language or decoupling tracking and mapping through multi-process parallel computing. Secondly, when dealing with dense mapping in large-scale environments, a more economical storage mode for neural points should be adopted, such as dynamically adjusting the density of neural points based on the complexity of the scene structure. Lastly, to address complex motions and challenging scenarios, the system's accuracy and robustness need further improvement. We believe that an organic integration of traditional geometric element constraints with SDF radiation field loss during pose tracking is a feasible approach.

VI. CONCLUSION

To achieve accurate simultaneous pose estimation and dense mapping, we propose a coupled LiDAR-Inertial Odometry system that integrates geometric kinematics with neural fields, (Semi-)KN-LIO. We fuse laser and inertial sensor information through an error-state Kalman filter. In the semi-coupled mode, the point cloud is first registered with the neural field, and then the error-state Kalman filter is updated with the composite registration result. In the tightly coupled mode, the system iteratively updates the error-state filter with the Kalman gain using residuals between the point cloud and the neural field until convergence. By coupling geometric kinematics with neural fields, our KN-LIO achieves comparable localization results to existing state-of-the-art LIO solutions on multiple public datasets and generates higher-quality dense mapping results than those of pure LiDAR solutions. Furthermore, support for multiple LiDARs enables our (Semi-)KN-LIO to easily fuse point cloud data from different perspectives.

REFERENCES

- [1] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LiDAR SLAM," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2016, pp. 1271–1278.
- [2] C. Qin, H. Ye, C. E. Pranata, J. Han, S. Zhang, and M. Liu, "LINS: A lidar-inertial state estimator for robust and efficient navigation," in *Proc. IEEE Int. Conf. Robot. Autom.*, Paris, France, May 2020, pp. 8899–8906.
- [3] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and R. Daniela, "LIO-SAM: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Las Vegas, USA, Oct. 2020, pp. 5135–5142.
- [4] Z. Wang, L. Zhang, Y. Shen, and Y. Zhou, "D-LIOM: Tightly-coupled direct lidar-inertial odometry and mapping," *IEEE Trans. Multimedia*, vol. 25, pp. 3905–3920, 2023.
- [5] K. Chen, R. Nemiroff, and B. T. Lopez, "Direct LiDAR-Inertial odometry: Lightweight LIO with continuous-time motion correction," *Proc. IEEE Int. Conf. Robot. Autom.*, pp. 3983–3989, 2023.
- [6] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2053–2073, 2022.
- [7] M. Kazhdan and H. Hoppe, "Screened poisson surface reconstruction," *ACM Trans. Graph.*, vol. 32, no. 3, Jul. 2013.
- [8] I. Vizzo, X. Chen, N. Chebrolu, J. Behley, and C. Stachniss, "Poisson surface reconstruction for lidar odometry and mapping," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021, pp. 5624–5630.
- [9] P. Labatut, J.-P. Pons, and R. Keriven, "Efficient multi-view reconstruction of large-scale scenes using interest points, delaunay triangulation and graph cuts," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2007, pp. 1–8.
- [10] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohi, J. Shotton, S. Hodges, and A. Fitzgibbon, "KinectFusion: Real-time dense surface mapping and tracking," in *Proc. IEEE Int. Sympo. Mixed Augmented Reality*, 2011, pp. 127–136.
- [11] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3d surface construction algorithm," *Siggraph Comput. Graph.*, vol. 21, no. 4, pp. 163–169, 1987.
- [12] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [13] X. Zhong, Y. Pan, J. Behley, and C. Stachniss, "SHINE-Mapping: Large-scale 3D mapping using sparse hierarchical implicit neural representations," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 8371–8377.
- [14] X. Yu, Y. Liu, S. Mao, S. Zhou, R. Xiong, Y. Liao, and Y. Wang, "NF-Atlas: Multi-volume neural feature fields for large scale lidar mapping," *IEEE Robot. Autom. Letters*, vol. 8, no. 9, pp. 5870–5877, 2023.
- [15] J. Deng, Q. Wu, X. Chen, S. Xia, Z. Sun, G. Liu, W. Yu, and L. Pei, "NeRF-LOAM: Neural implicit representation for large-scale incremental lidar odometry and mapping," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 8184–8193.
- [16] Y. Pan, X. Zhong, L. Wiesmann, T. Posewsky, J. Behley, and C. Stachniss, "PIN-SLAM: LiDAR SLAM using a point-based implicit neural representation for achieving global map consistency," *IEEE Trans. Robot.*, vol. 40, pp. 4045–4064, 2024.
- [17] J. Zhang and S. Singh, "LOAM: Lidar odometry and mapping in real-time," in *Proc. Robot. Sci. Syst. Conf.*, 2014, pp. 1–9.
- [18] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, 2018.
- [19] H. Ye, Y. Chen, and M. Liu, "Tightly coupled 3D lidar inertial odometry and mapping," in *Proc. IEEE Int. Conf. Robot. Autom.*, Montreal, Canada, May 2019, pp. 3144–3150.
- [20] P. Besl and N. D. McKay, "A method for registration of 3-D shapes," *IEEE Trans. Patt. Anal. Mach. Intell.*, vol. 14, no. 2, pp. 239–256, 1992.
- [21] J. Sola, "Quaternion kinematics for the error-state Kalman filter," 2020, arXiv preprint arXiv:1711.02508.
- [22] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2018, pp. 4758–4765.
- [23] M. Kaess, A. Ranganathan, and F. Dellaert, "iSAM: Incremental smoothing and mapping," *IEEE Trans. Robot.*, vol. 24, no. 6, pp. 1365–1378, 2008.
- [24] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. Leonard, and F. Dellaert, "iSAM2: Incremental smoothing and mapping with fluid relinearization and incremental variable reordering," in *Proc. IEEE Int. Conf. Robot. Autom.*, Shanghai, China, May 2011, pp. 3281–3288.
- [25] J. Lv, K. Hu, J. Xu, Y. Liu, X. Ma, and X. Zuo, "CLINS: Continuous-time trajectory estimation for lidar-inertial system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Prague, Czech Republic, Sept. 2021, pp. 6657–6663.
- [26] Z. Wang, L. Zhang, S. Zhao, and Y. Zhou, "Ct-LVI: A framework toward continuous-time laser-visual-inertial odometry and mapping," *IEEE Trans. Circuits Syst. Video Tech.*, vol. 34, no. 6, pp. 4378–4391, 2024.
- [27] W. Xu and F. Zhang, "FAST-LIO: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robot. Autom. Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.
- [28] J. Chen, D. Bautembach, and S. Izadi, "Scalable real-time volumetric surface reconstruction," *ACM Trans. Graph.*, vol. 32, no. 4, pp. 1–16, 2013.
- [29] M. Nießner, M. Zollhöfer, S. Izadi, and M. Stamminger, "Real-time 3D reconstruction at scale using voxel hashing," *ACM Trans. Graph.*, vol. 32, no. 6, pp. 1–11, 2013.
- [30] O. Köhler, V. Prisacariu, J. Valentin, and D. Murray, "Hierarchical voxel block hashing for efficient integration of depth images," *IEEE Robot. Autom. Letters*, vol. 1, no. 1, pp. 192–197, 2016.
- [31] E. Vespa, N. Nikolov, M. Grimm, L. Nardi, P. H. J. Kelly, and S. Leutenegger, "Efficient octree-based volumetric SLAM supporting signed-distance and occupancy mapping," *IEEE Robot. Autom. Letters*, vol. 3, no. 2, pp. 1144–1151, 2018.
- [32] D. Lefloch, M. Kluge, H. Sarbolandi, T. Weyrich, and A. Kolb, "Comprehensive use of curvature for robust and accurate online surface reconstruction," *IEEE Trans. Patt. Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2349–2365, 2017.
- [33] D. Lefloch, T. Weyrich, and A. Kolb, "Anisotropic point-based fusion," in *Proc. Int. Conf. Infor. Fusion*, 2015, pp. 2121–2128.
- [34] J. Lin, C. Yuan, Y. Cai, H. Li, Y. Ren, Y. Zou, X. Hong, and F. Zhang, "ImMesh: An immediate LiDAR localization and meshing framework," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4312–4331, 2023.
- [35] Z. Zhu, S. Peng, V. Larsson, W. Xu, H. Bao, Z. Cui, M. R. Oswald, and M. Pollefeys, "NICE-SLAM: Neural implicit scalable encoding for SLAM," in *Proc. IEEE Conf. Comput. Vis. Patt. Recog.*, 2022, pp. 12776–12786.
- [36] Z. Zhu, S. Peng, V. Larsson, Z. Cui, M. R. Oswald, A. Geiger, and M. Pollefeys, "NICER-SLAM: Neural implicit scene encoding for RGB SLAM," in *Proc. Int. Conf. 3D Vision*, 2024, pp. 42–52.
- [37] X. Yang, H. Li, H. Zhai, Y. Ming, Y. Liu, and G. Zhang, "Vox-Fusion: dense tracking and mapping with voxel-based neural implicit representation," in *IEEE Int. Symp. Mixed Augmented Reality*, 2022, pp. 499–507.
- [38] Q. Xu, Z. Xu, J. Philip, S. Bi, Z. Shu, K. Sunkavalli, and U. Neumann, "Point-NeRF: Point-based neural radiance fields," in *Proc. IEEE Conf. Comput. Vis. Patt. Recog.*, 2022, pp. 5428–5438.
- [39] E. Sandström, Y. Li, L. Van Gool, and M. R. Oswald, "Point-SLAM: Dense neural point cloud-based SLAM," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 18387–18398.
- [40] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 1–14, 2023.
- [41] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, "Gaussian splatting SLAM," in *Proc. IEEE Conf. Comput. Vis. Patt. Recog.*, 2024, pp. 18039–18048.
- [42] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "SplaTAM: Splat, track map 3D Gaussians for dense RGB-D SLAM," in *Proc. IEEE Conf. Comput. Vis. Patt. Recog.*, 2024, pp. 21357–21366.
- [43] X. Gao, *SLAM in Autonomous Driving*. Electronic Industry Press, 2023.
- [44] C. Bai, T. Xiao, Y. Chen, H. Wang, F. Zhang, and X. Gao, "Faster-LIO: Lightweight tightly coupled lidar-inertial odometry using parallel sparse incremental voxels," *IEEE Robot. Autom. Letters*, vol. 7, no. 2, pp. 4861–4868, 2022.
- [45] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Trans. Patt. Anal. Mach. Intell.*, vol. 13, no. 4, pp. 376–380, 1991.
- [46] A. Knapitsch, J. Park, Q.-Y. Zhou, and V. Koltun, "Tanks and temples: benchmarking large-scale scene reconstruction," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–13, 2017.
- [47] J. Ruan, B. Li, Y. Wang, and Y. Sun, "SLAMesh: Real-time LiDAR simultaneous localization and meshing," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 3546–3552.
- [48] I. Vizzo, T. Guadagnino, J. Behley, and C. Stachniss, "VDBFusion: flexible and efficient TSDF integration of range sensor data," *Sensors*, vol. 22, no. 3, pp. 1296–1310, 2022.

- [49] J. Huang, Z. Gojcic, M. Atzmon, O. Litany, S. Fidler, and F. Williams, "Neural kernel surface reconstruction," in *Proc. IEEE Conf. Comput. Vis. Patt. Recog.*, 2023, pp. 4369–4379.
- [50] I. Vizzo, T. Guadagnino, B. Mersch, L. Wiesmann, J. Behley, and C. Stachniss, "KISS-ICP: in defense of point-to-point ICP-simple, accurate, and robust registration if done the right way," *IEEE Robot. Autom. Letters*, vol. 8, no. 2, pp. 1029–1036, 2023.